

AI-Powered Enterprise E-Commerce Platform

Monitoring and Logging Architecture Specification

Document Information

Item	Description
Project Name	AI-Powered Enterprise E-Commerce Platform
Document Type	Monitoring and Logging Architecture
Version	1.0
Prepared By	Project Team
Status	Draft

Revision History

Version	Date	Description
1.0	DD/MM/YYYY	Initial Version

Table of Contents

- 1. Introduction
- 2. Purpose
- 3. Objectives
- 4. Observability Principles
- 5. Monitoring Architecture
- 6. Logging Architecture
- 7. Metrics Collection
- 8. Application Monitoring
- 9. Infrastructure Monitoring
- 10. Database Monitoring
- 11. AI Model Monitoring
- 12. Alerting Strategy
- 13. Dashboards
- 14. Incident Management
- 15. Retention Policies
- 16. Future Enhancements

17. Conclusion

18. References

1. Introduction

Modern distributed systems require comprehensive observability mechanisms.

The AI-Powered Enterprise E-Commerce Platform consists of multiple services:

- Frontend
- Spring Boot Services
- FastAPI AI Services
- PostgreSQL
- Redis
- RabbitMQ
- Docker Infrastructure

Monitoring and logging help ensure:

- Reliability
- Performance
- Security
- Availability
- Troubleshooting capability

2. Purpose

The purpose of this document is to:

- Define monitoring mechanisms.
- Establish logging standards.
- Improve incident detection.
- Support system debugging.
- Enable proactive maintenance.

3. Objectives

MON-01

Detect failures quickly.

MON-02

Monitor application performance.

MON-03

Provide centralized logs.

MON-04

Track AI model health.

MON-05

Support incident response.

MON-06

Improve operational visibility.

4. Observability Principles

The architecture follows three pillars of observability:

Metrics

Numerical measurements over time.

Examples:

- CPU usage
 - Response time
 - Error rates
-

Logs

Detailed event records.

Examples:

- Login events
 - Exceptions
 - API requests
-

Traces (Future Enhancement)

Track request flow across services.

Examples:

- User Request
 - Backend Service
 - AI Service
 - Database Query
-

5. High-Level Monitoring Architecture

```
graph TD;
  A[Application Services] --> B[Metrics Exporters];
  B --> C[Prometheus];
  C --> D[Grafana Dashboards];
```

Logging Architecture

```
graph TD;
  A[Services] --> B[Log Collectors];
  B --> C[Elasticsearch];
  C --> D[Kibana];
```

6. Monitoring Tools

Metrics Collection

Technology:

Prometheus

Responsibilities:

- Scrape metrics
 - Store time-series data
-

Visualization

Technology:

Grafana

Responsibilities:

- Dashboards
 - Alert visualization
-

Logging

Technology:

ELK Stack

Components:

- Elasticsearch
 - Logstash
 - Kibana
-

Future Enhancements

- OpenTelemetry
 - Jaeger Distributed Tracing
-

7. Metrics Collection Strategy

Metrics should be collected from:

Frontend

Examples:

- Page load time
 - Client-side errors
 - API latency
-

Backend Services

Examples:

- Request count
 - Error count
 - JVM memory usage
 - Response time
-

AI Services

Examples:

- Prediction latency
 - Inference count
 - Model performance metrics
-

Databases

Examples:

- Active connections
 - Slow queries
 - Query execution time
-

8. Application Monitoring

Authentication Service

Monitor:

- Login attempts
 - Failed logins
 - Token generation rate
-

Product Service

Monitor:

- Search requests
 - Product retrieval latency
-

Order Service

Monitor:

- Orders created
 - Failed transactions
-

Inventory Service

Monitor:

- Stock updates
 - Inventory synchronization failures
-

Notification Service

Monitor:

- Email failures
 - Queue delays
-

9. Infrastructure Monitoring

Docker Monitoring

Metrics:

- Container CPU usage
 - Container memory usage
 - Container restart count
-

Server Monitoring

Metrics:

- CPU utilization
 - Memory usage
 - Disk usage
 - Network throughput
-

Network Monitoring

Metrics:

- Request throughput
 - Packet loss
 - Latency
-

10. Database Monitoring

PostgreSQL Metrics

Monitor:

- Active connections
 - Query latency
 - Deadlocks
 - Slow queries
 - Database size
-

Redis Metrics

Monitor:

- Memory usage
 - Cache hit ratio
 - Evictions
-

RabbitMQ Metrics

Monitor:

- Queue length
 - Message processing rate
 - Failed messages
-

11. Logging Architecture

Logging should be centralized.

Logging Levels

TRACE

Detailed execution information.

DEBUG

Development information.

INFO

Normal application events.

WARN

Potential issues.

ERROR

Failures requiring attention.

FATAL

Critical failures.

Log Format

Recommended format:

```
{
  "timestamp": "",
  "level": "",
  "service": "",
  "traceId": "",
  "message": ""
}
```

Logging Standards

Every log should include:

- Timestamp
- Service Name
- Request ID

- User ID (when possible)
 - Severity Level
-

12. Security Logging

Log:

- Failed login attempts
 - Authorization failures
 - Suspicious requests
 - Administrative actions
 - API abuse attempts
-

Retention

Security logs:

90–180 Days

13. AI Model Monitoring

AI systems require additional monitoring.

Recommendation System Monitoring

Monitor:

- Recommendation latency
 - Recommendation acceptance rate
 - Click-through rate
-

Fraud Detection Monitoring

Monitor:

- Prediction counts
 - False positives
 - False negatives
-

Forecasting Monitoring

Monitor:

- Forecast accuracy
 - Prediction drift
-

Model Drift Monitoring

Monitor:

- Input data changes
 - Performance degradation
 - Feature distribution changes
-

AI Metrics Examples

Metric	Purpose
Precision	Recommendation quality
Recall	Coverage
F1 Score	Fraud performance
RMSE	Forecast quality

14. Alerting Strategy

Alerts should be generated when thresholds are exceeded.

Critical Alerts

Examples:

- Service unavailable
 - Database failure
 - Queue failure
-

Warning Alerts

Examples:

- High CPU usage

- Increased latency
- Elevated error rates

Example Thresholds

Metric	Threshold
CPU Usage	>80%
Memory Usage	>85%
API Latency	>2 sec
Error Rate	>5%
Queue Length	Above threshold

Alert Channels

Possible mechanisms:

- Email
- Slack
- Microsoft Teams

15. Dashboards

Recommended dashboards:

System Dashboard

Display:

- Overall health
- Service availability

Backend Dashboard

Display:

- API metrics
- Error rates

Database Dashboard

Display:

- Connections
- Query performance

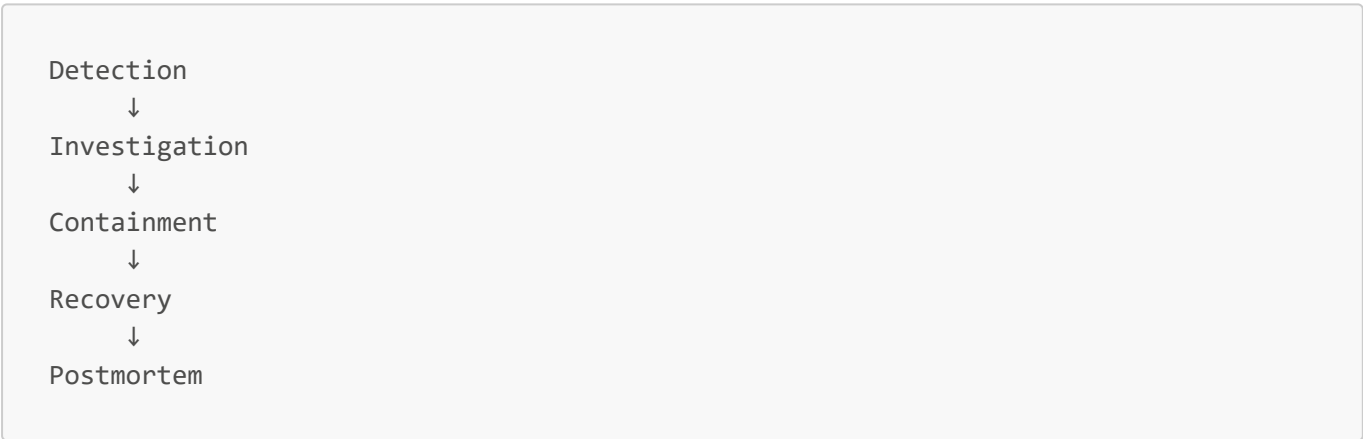
AI Dashboard

Display:

- Model metrics
- Drift indicators
- Inference latency

16. Incident Management

Incident Lifecycle



Incident Priorities

Priority	Description
P1	System Down
P2	Major Feature Failure
P3	Partial Service Failure
P4	Minor Issue

Post-Incident Activities

- Root Cause Analysis
- Documentation
- Preventive Actions

17. Retention Policies

Metrics Retention

Recommended:

30–90 Days

Application Logs

Recommended:

30–90 Days

Audit Logs

Recommended:

90–180 Days

Backup Logs

Recommended:

1 Year

18. Monitoring Risks and Mitigation

Risk	Mitigation
Missing Metrics	Standard instrumentation
Excessive Logs	Retention policies
Alert Fatigue	Proper thresholds
Storage Growth	Log rotation

19. Future Enhancements

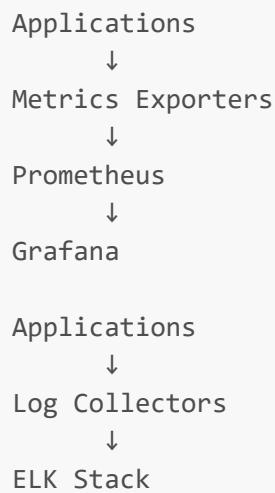
Possible improvements:

- OpenTelemetry
- Distributed Tracing
- Jaeger
- Service Mesh Observability
- AI-Powered Anomaly Detection
- Predictive Monitoring
- Chaos Engineering

20. Recommended Directory Structure

```
monitoring/
├── prometheus/
│   └── prometheus.yml
├── grafana/
│   ├── dashboards/
│   └── datasources/
├── elk/
│   ├── elasticsearch/
│   ├── logstash/
│   └── kibana/
├── alerts/
│   └── alert_rules.yml
```

21. Recommended Architecture Diagram



22. Conclusion

The monitoring and logging architecture provides:

- Operational visibility
- Rapid incident detection
- Improved troubleshooting
- AI model observability
- Better reliability and maintainability

The architecture supports future enterprise growth and cloud-native operations while ensuring the platform remains observable and manageable.

References

- [1] Prometheus Documentation.
- [2] Grafana Documentation.
- [3] Elasticsearch Documentation.
- [4] OpenTelemetry Documentation.
- [5] Google Site Reliability Engineering (SRE).
- [6] Newman, S. Building Microservices.
- [7] NIST Logging Recommendations.